Google Summer of Code 2026

Proposal

LabLua

Lunatik eBPF Abstraction Layer

Bindings for Linux Traffic Control, Scheduler and eBPF Maps

Ashwani Kumar Kamal

Email: ashwanikamal.im421@gmail.com
GitHub: https://github.com/sneaky-potato/

1. Basics

Preferred Email	ashwanikamal.im421@gmail.com
GitHub	https://github.com/sneaky-potato/
Matrix ID	@sneaky-potato:matrix.org
Academic Background	I graduated from IIT Kharagpur in 2024, with Bachelor of Technology in Computer Science and Engineering
Time Commitments During GSoC	I can dedicate 30-35 hours per week to the project. I do have a day job but I will be able to take time out to do this. My working hours are flexible, and I intend to use my off times and weekends for GSoC.

2. Experience

2.1. Programming Languages

I primarily work with languages including C, C++, Go, and Lua. I also use Bash for scripting.

2.2. Tools for development

My development workflow uses Git, Linux-based environments, and the Neovim editor. I also use some debugging tooling such as GDB, strace, ltrace, libbpf utilities, bpftool.

2.3. Lua Experience

I got introduced to Lua via Neovim configuration and basic scripting. I contributed to Lunatik through which I got the major experience of Lua and Lua C API. It helped me understand how Lua fits as an embeddable language.

2.4. Software Development Experience

I work as a Software Engineer at **Schlumberger** where I build backend services and internal tools used in production systems. My work involves designing microservices, optimizing database access patterns, and improving the reliability and performance of distributed systems.

Recently, I led the migration of over 1 million customer records to separate sensitive personally identifiable information (PII) from operational data, improving data isolation and security. I also optimized identity lookup queries using improved SQL joins and Redis caching, reducing P95 request latency by approximately 40%.

I regularly collaborate with other engineers using Git-based workflows including pull requests, code reviews, and issue-driven development. I am comfortable discussing trade-offs, iterating on reviews, and maintaining documentation alongside code.

Prior to this role, I completed a software engineering internship at **Piramal Retail Finance** where I built a prototype in house job scheduler capable of handling more than 10,000 scheduled tasks per day across multiple microservices.

2.5. Previous Projects

Some technical projects I have built include:

- **Goof Programming Language** (Hobby project):

As an attempt to understand language runtimes, I wrote a compiler for a programming language Goof which is a stack-based, concatenative language inspired by Forth for x86_64 architectures.

The compiler, emits x86_64 assembly and produces statically linked ELF binaries for Linux. The project includes a lexer, parser, stack-based intermediate representation, type checking, and a runtime supporting control flow, procedures, and system calls.

- **Route Optimization Backend (Inter IIT Tech Meet)** (Competition):

Developed backend services for a delivery route planning system used during the Inter IIT Tech Meet competition. The system exposed HTTP and gRPC APIs, used RabbitMQ for asynchronous task processing, and was containerized with Docker to enable reliable deployment.

These projects helped me develop strong experience working with systems-level programming, distributed systems, and backend service design.

2.6. Open Source Contributions

I have been actively exploring and contributing to the Lunatik ecosystem, which enables Lua scripting within the Linux kernel.

While studying the existing implementation of **Lunatik** modules, I worked on improving the method dispatch mechanism used by Lua objects in the kernel runtime. By replacing repeated table lookups with eager closure wrapping, I reduced method call overhead from roughly 275ns to 98ns in benchmark tests across one million iterations.

Related PRs:
- #410: wrap class methods eagerly instead of via __index
- #434: add support for non-shared objects in shared class

In addition to Lunatik, I have also contributed patches to the **etcd** project, focusing on improving the correctness and determinism of the robustness testing framework by removing redundant delete and compact operations.

3. GSoC

Participated in GSoC before?	No
Applied but not selected before?	No
Applied to other organizations this year?	No

4. Project

I started out with the project **Lunatik Binding for Linux Traffic Control (TC) and eBPF Maps** which is from the idea list.

I noticed while contributing, that to run Lua callbacks from bpf programs there was one hook `bpf_luaxdp_run` but it was tightly coupled to XDP and that the abstraction layer was missing.

While doing the technical reading required for this project I realized how Lua could be used as a high level abstraction to further increase the expressiveness of eBPF programs.

Hence I propose **Lunatik eBPF Abstraction Layer** which targets multiple Linux subsystems.

4.1. Selected Idea

This project builds on Lunatik's existing networking capabilities, specifically the `luaxdp` module, and proposes a **generalized eBPF integration layer**.

The motivation comes from studying Lunatik's current architecture and realizing that the value of new bindings is **combinatory rather than linear**. Each additional subsystem integrated with Lua multiplies the scripting possibilities available to the kernel.

Therefore, instead of implementing a single new binding in isolation, this project proposes building a reusable infrastructure layer that future bindings can use.

4.2. Project Description

4.2.1. The Problem

eBPF has become the dominant mechanism for extending the Linux kernel at runtime. It is fast, safe, and verifier-enforced. However, the verifier also limits expressiveness and complex logic such as:

- dynamic string matching
- pattern-based rules
- dynamic policy tables
- hot-swappable logic

Lua provides a convenient way to express dynamic policies that are difficult to encode within the verifier constraints of eBPF. It is small, embeddable, and designed to act as a scripting layer that extends a host system without replacing its core architecture.

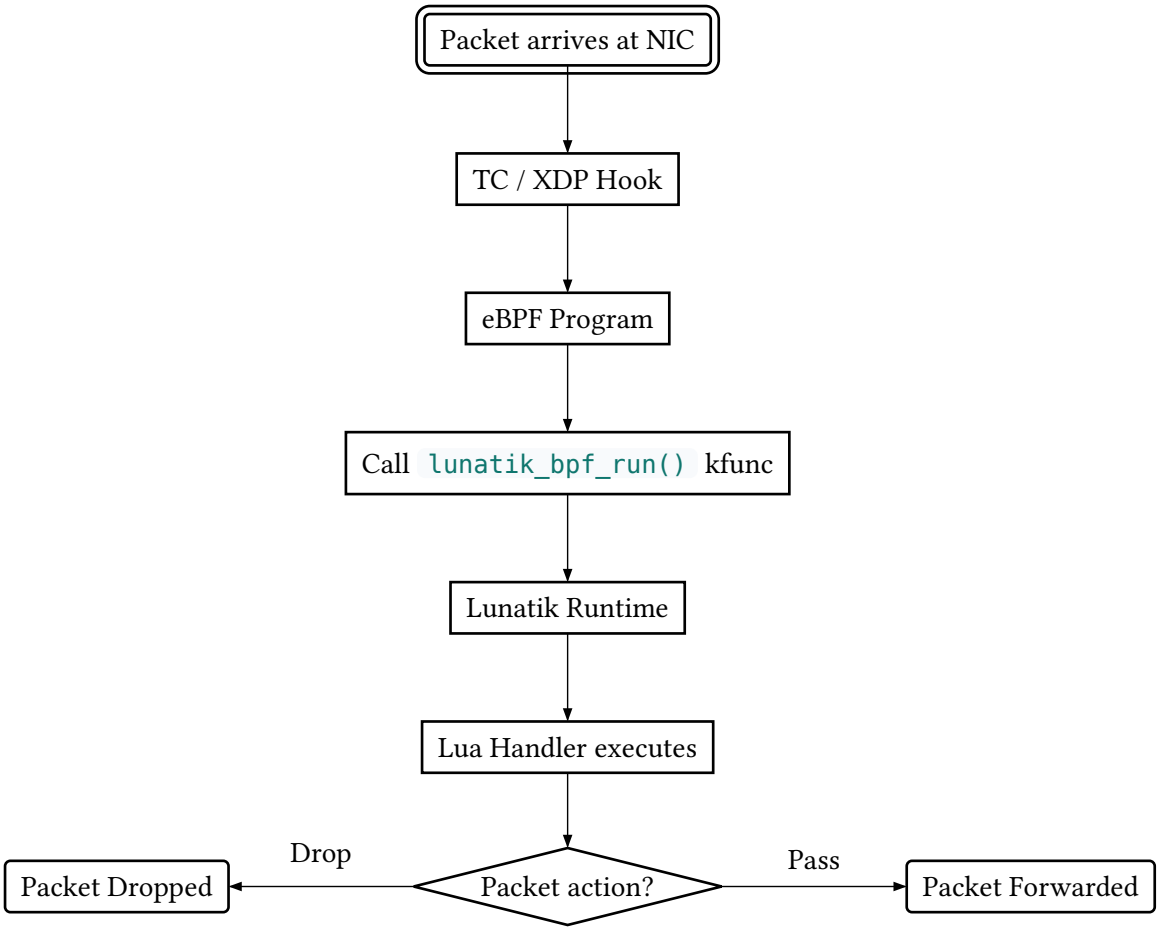
4.2.2. Design Philosophy

This project follows a simple design pattern:

- **eBPF defines structure and safe hooks: fast path**
- **Lua implements dynamic policy logic: slow path**

The idea is to allow eBPF programs to delegate complex decisions to Lua handlers inside Lunatik.

The architecture looks like this:



The Lua handler evaluates policy and returns a verdict, after which execution continues inside the eBPF program.

Presently `luaxdp` implements this pattern but it is **tightly coupled to XDP**. The kfunc `bpf_luaxdp_run()` is registered only for `BPF_PROG_TYPE_XDP` and receives an `xdp_md` context.

This project aims to generalize that design.

4.3. Core Components

4.3.1. Generic `lunatik_bpf_run()` Layer

Since eBPF programs cannot call arbitrary kernel functions, the Linux kernel provides kfuncs as a mechanism for exposing functionality to BPF programs. These functions are registered with the BPF subsystem using BTF-based kfunc registration, which allows the verifier to enforce safety constraints.

The eBPF docs mention a list of kfuncs here: https://docs.ebpf.io/linux/kfuncs/, This could provide with inspiration for future kfunc support by Lunatik.

4.3.1.1. Kfunc registration

```
__bpf_kfunc int bpf_luaxdp_run(struct xdp_md *ctx, ...);
__bpf_kfunc int bpf_luatc_run(struct __sk_buff *skb, ...);
```

```
__bpf_kfunc int bpf_luasched_run(struct task_struct *task, struct
luasched_result *result, ...);
```

The functions are then registered with the BPF subsystem.

```
BTF_KFUNCS_START(bpf_luatc_set)
BTF_ID_FLAGS(func, bpf_luatc_run)
BTF_KFUNCS_END(bpf_luatc_set)
```

Finally, during module initialization, the kfunc set is registered with the kernel for the relevant BPF program type:

```
register_btf_kfunc_id_set(
    BPF_PROG_TYPE_SCHED_CLS,
    &bpf_luatc_kfunc_set
);
```

This restricts them to specific program types such as `BPF_PROG_TYPE_SCHED_CLS`.

4.3.1.2. Generic interface

Shared internal function that any type-specific kfunc can call:

```
typedef int  (*lunatik_bpf_input_cb)(lua_State *L, void *ctx);
typedef void (*lunatik_bpf_output_cb)(lua_State *L, void *result);

int lunatik_bpf_run(
    const char           *runtime,
    lunatik_bpf_input_cb  push_ctx,    // pushes input context onto Lua stack
    void                 *ctx,
    lunatik_bpf_output_cb read_result, // reads output from stack into result
    void                 *result       // caller-allocated result buffer
);
```

This function:

1. Looks up the named Lunatik runtime
2. Calls `push_ctx(L, ctx)` to push the context onto the Lua stack as a typed userdata
3. Invokes the registered Lua handler
4. Calls `read_result(L, result)` to read however many return values the subsystem expects off the Lua stack into the caller-allocated buffer
5. Cleans the Lua stack

Type-specific kfuncs then become thin wrappers:

```
// bpf_luaxdp_run will get refactored, behaviour unchanged
__bpf_kfunc int bpf_luaxdp_run(struct xdp_md *ctx, ...)
{
```

```
    struct luaxdp_result result = { .action = XDP_PASS };
    lunatik_bpf_run(runtime, luaxdp_push_ctx, ctx, luaxdp_read_result,
&result);
    return result.action;
}

// bpf_luatc_run will be a new addition
__bpf_kfunc int bpf_luatc_run(struct __sk_buff *skb, ...)
{
    struct luatc_result result = { .action = TC_ACT_OK };
    lunatik_bpf_run(runtime, luatc_push_ctx, skb, luatc_read_result, &result);
    return result.action;
}

// bpf_luasched_run will be a new addition
__bpf_kfunc int bpf_luasched_run(struct task_struct *task, struct
luasched_result *result)
{
    lunatik_bpf_run(runtime, luasched_push_ctx, task, luasched_read_result,
result);
    return 0;
}
```

The `runtime` parameter allows multiple Lua runtimes to coexist, enabling different programs to delegate processing to different Lua handlers.

Each kfunc is registered for its specific `bpf_prog_type` via `BTF_KFUNCS_START` / `BTF_KFUNCS_END`, maintaining verifier safety.

4.3.2. Traffic Control Binding (luatc)

Traffic Control is the primary demonstration of the generic layer. TC operates on `__sk_buff` and is the place in the kernel with several capabilities:

- `tc_classid` : direct assignment to HTB qdisc classes for traffic shaping
- `tstamp` : packet transmit timestamp for pacing
- `priority` , `tc_index` , `mark` : classification metadata

None of these are available at XDP. TC is where shaping decisions are made. Lua is where complex policy logic: string matching, pattern tables, dynamic rules, is expressed.

We can extend the recently merged `luaskb` module which provides an abstraction to `__sk_buff` data structure:

```
skb.mark          -- skb->mark           (r/w)
skb.priority      -- skb->priority       (r/w)
skb.tc_index      -- skb->tc_index       (r/w)
skb.tc_classid    -- skb->tc_classid     (r/w)
skb.protocol      -- skb->protocol       (r)
skb.tstamp        -- skb->tstamp         (r/w)
```

```lua
local tc = require("tc")

tc.action.OK          -- TC_ACT_OK         = 0
tc.action.RECLASSIFY  -- TC_ACT_RECLASSIFY = 1
tc.action.SHOT        -- TC_ACT_SHOT       = 2
tc.action.PIPE        -- TC_ACT_PIPE       = 3
tc.action.STOLEN      -- TC_ACT_STOLEN     = 4
tc.action.REDIRECT    -- TC_ACT_REDIRECT   = 7

tc.attach(handler)    -- register Lua callback
tc.detach()           -- unregister
```

The Lua handler returns a single integer, the TC verdict. `luatc_read_result` reads `lua_tointeger(L, -1)`, making the output path as thin as possible while remaining consistent with the generic interface used by all subsystems.

4.3.3. Scheduler Binding (luasched)

Linux kernel 6.12 introduced sched_ext (extensible scheduler) as a new scheduling class that allows pluggable CPU schedulers via eBPF.

This enables implementing and dynamically loading thread schedulers. No need for recompiling the kernel and rebooting.

sched-ext/scx project is a collection of sched_ext schedulers and tools.

With the new generic `lunatik_bpf_run` layer we can start targeting other subsystems and scheduler is the primary fit.

sched_ext operates mostly around `struct task_struct` and we will need to write the bindings for this on Lua side.

```lua
local task = require("sched.task")

task.pid    -- task_struct->pid       (r)
task.comm   -- task_struct->comm      (r)
task.prio   -- task_struct->prio      (r/w)
```

```lua
local sched = require("sched")

sched.dsq.GLOBAL          -- SCX_DSQ_GLOBAL
sched.dsq.LOCAL           -- SCX_DSQ_LOCAL

sched.slice.DEFAULT       -- SCX_SLICE_DFL
sched.slice.BYPASS        -- SCX_SLICE_BYPASS

sched.attach(handler)     -- register Lua callback
sched.detach()            -- unregister
```

The Lua handler returns two values: {dsq, slice}. `luasched_read_result` reads these values into `luasched_result`.

4.3.4. eBPF Maps Module

To fully use the bpf ecosystem via Lunatik, we need access to shared kernel state. Hence support for **eBPF maps module for Lunatik** is important, this will allow Lunatik to have:

- **Stateful policies**: a Lua handler writes `{ip -> classid}` to a map; the eBPF fast path reads it
- **Cross-binding composition**: `luaxdp` writes to a map that `luatc` reads, enabling pipelines that span subsystems
- **Operator visibility**: Lua scripts read kernel maps to inspect state without a separate userspace tool

```lua
local map = require("ebpf.map")

local flow_table = map.open("/sys/fs/bpf/flow_cache")
local classid = flow_table:lookup(dst_ip)
flow_table:update(src_ip, new_classid)
flow_table:delete(stale_ip)
```

eBPF maps are created via `bpf(BPF_MAP_CREATE, ...)` and accessed via `bpf_map_lookup_elem`, `bpf_map_update_elem`, and `bpf_map_delete_elem` helpers. We want to access these via kernel's internal map API without going through syscall interface.

The following APIs are exposed by kernel to use:

- `bpf_map_get(u32 ufd)`
- `bpf_map_get_with_uref(u32 ufd)`

These will need the file descriptor which needs to be created via a userspace process context.

4.3.4.1. Map Creation

This design requires bpf maps to be created outside Lunatik, via userspace programs like `bpftool`. Maps are created in userspace and pinned to bpffs, allowing them to persist independently and be shared across subsystems.

```
bpftool map create /sys/fs/bpf/flow_cache type hash key 4 value 4 entries 128
name flow_cache
```

4.3.4.2. Map Lookup and usage

Map resolution normally happens as follows:

- from userspace we call open(path) to get an fd on the pinned bpffs file.
- now we share this fd to kernel side
- we can call `bpf_map_get_with_uref(u32 ufd)` from kernel side to resolve the fd to pointer to `struct bpf_map`

This approach will require adding a userspace bpf helper method to be called from `/bin/lunatik`. Hence changes might be required in both `/bin/lunatik` and `driver.lua`

The other approach, bypasses using the standard way and uses kernel helpers to resolve map pointer directly from bpffs path. We can use `kern_path` to resolve the dentry without triggering the open handler, then read `inode->i_private` directly where bpffs stores the pointer to `struct bpf_map`.

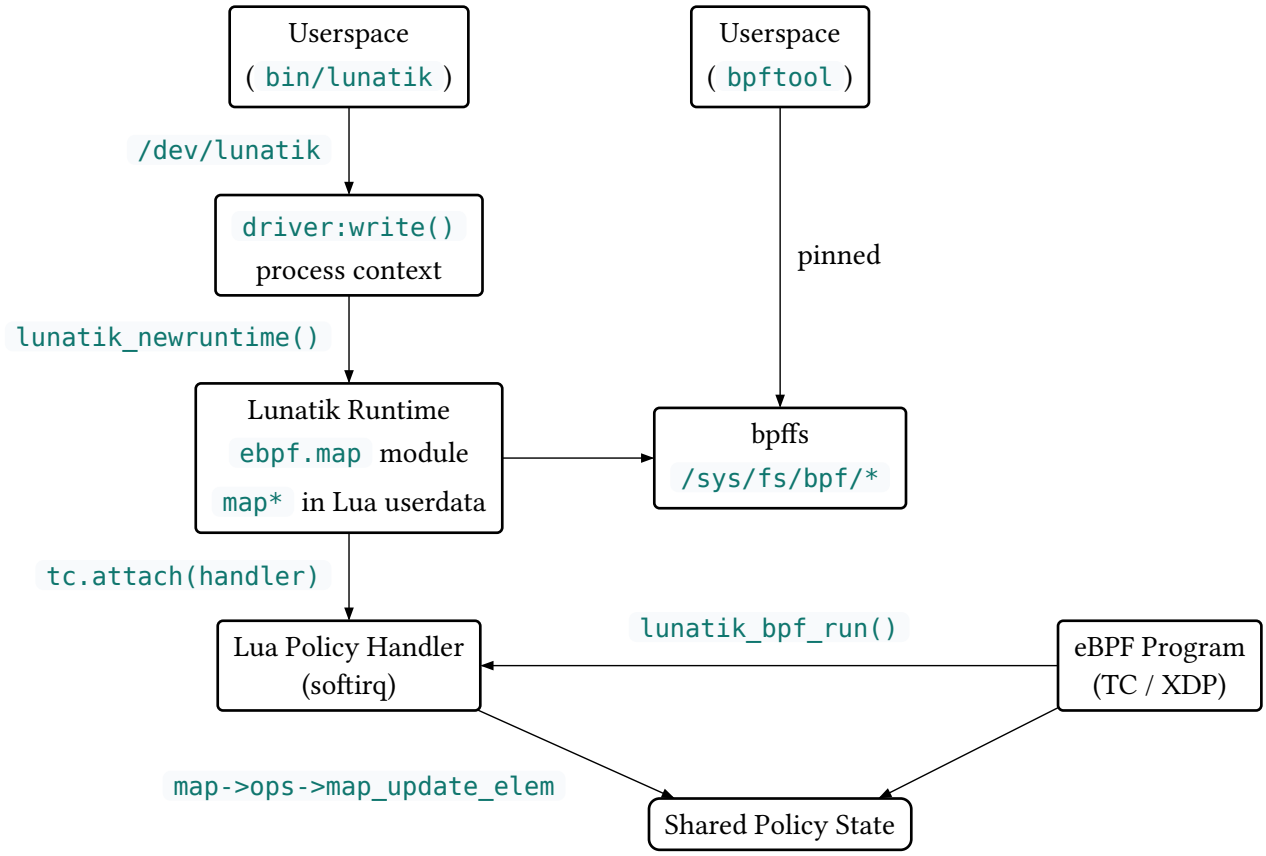
This has been verified via a kernel module POC.

The resolved pointer to `struct bpf_map` is stored in a Lua userdata and reused by all subsequent lookup/update/delete calls. These are safe to call from softirq.

- `lookup(key)` on the map pointer will return the value stored in the map for the provided key. We can reuse `luadata` for the actual types.
- `update(key, data)` on the map pointer will update the map for the provided key with the given data.
- `delete(key)` on the map pointer will delete the key from the map.
- Once we have the pointer to `struct bpf_map`, we could call kernel ops helpers defined here to do lookup, update, delete.

4.3.4.3. Cleanup

- `close()` will cleanup the map from Lua, and decrease the reference counter via `bpf_map_put(struct bpf_map *map)`



4.3.5. First Packet Classification

This demo will use eBPF TC classifier to check if a packet is SNI, then invoke the Lua callback and classify the packet and cache the result in a map.

1. Setup TC setup with classes for separate rates

```
tc qdisc del dev eth0 root 2>/dev/null
tc qdisc add dev eth0 root handle 1: htb default 30

tc class add dev eth0 parent 1: classid 1:10 htb rate 20mbit  # realtime
tc class add dev eth0 parent 1: classid 1:20 htb rate 10mbit  # streaming
tc class add dev eth0 parent 1: classid 1:30 htb rate 5mbit   # bulk/default

# Attach the below eBPF program on EGRESS
tc filter add dev eth0 parent 1: bpf da obj tc.bpf.o sec classifier
```

2. eBPF program

```
extern int bpf_luatc_run(char *key, size_t key__sz, struct __sk_buff *skb,
void *arg, size_t arg__sz) __ksym;

SEC("tc")
int tls_classifier(struct __sk_buff *skb)
{
    struct flow_key key = extract_key(skb);
    // key could be made from src_ip, dst_ip, src_port, dst_port, ip_proto
    // We could also timestamp the cache entries to handle TCP port reuse.

    __u32 *classid = bpf_map_lookup_elem(&flow_cache, &key);
    if (classid) {
        skb->tc_classid = *classid;
        return TC_ACT_OK;
    }

    // only invoke Lua if this looks like a TLS ClientHello
    if (!is_tls_client_hello(skb))
        return TC_ACT_OK;

    return bpf_luatc_run(runtime, sizeof(runtime), skb, NULL, 0);
}
```

3. Lua policy handler

```
local tc  = require("tc")
local map = require("ebpf.map")

local cache = map.open("/sys/fs/bpf/flow_cache")

local policy = {
    ["meet%.google%.com$"]  = 0x00010010,  -- realtime
    ["%.zoom%.us$"]         = 0x00010010,  -- realtime
    ["netflix%.com$"]       = 0x00010020,  -- streaming
}

local function parse_sni(p)
```

```lua
    -- get sni from the packet
end

local function handler(p)
    local sni = parse_sni(p)
    if not sni then return tc.action.OK end

    for pattern, classid in pairs(policy) do
        if sni:match(pattern) then
            cache:update(flow_key(p), classid)
            p.tc_classid = classid
            return tc.action.OK
        end
    end

    return tc.action.OK
end

tc.attach(handler)
```

4.3.6. Workload scheduling

This example will use eBPF program invoking Lua for implementing the enqueue logic with the help of maps. Lua should not be invoked on every scheduling decision; it is restricted to cold-path classification with results cached in BPF maps.

1. Define queues and dispatch logic

```ebpf
#define DSQ_REALTIME  0
#define DSQ_BATCH     1
#define DSQ_DEFAULT   2

s32 BPF_STRUCT_OPS(luasched_init)
{
    scx_bpf_create_dsq(DSQ_REALTIME, -1);
    scx_bpf_create_dsq(DSQ_BATCH,    -1);
    scx_bpf_create_dsq(DSQ_DEFAULT,  -1);
    return 0;
}

void BPF_STRUCT_OPS(luasched_dispatch, s32 cpu, struct task_struct *prev)
{
    /* drain realtime first, then batch, then default */
    if (!scx_bpf_consume(DSQ_REALTIME))
        if (!scx_bpf_consume(DSQ_BATCH))
            scx_bpf_consume(DSQ_DEFAULT);
}
```

2. Implement hooks in eBPF and call `bpf_luasched_run` kfunc

```
extern int bpf_luasched_run(struct task_struct *p, struct luasched_result
*result) __ksym;

void BPF_STRUCT_OPS(luasched_enqueue, struct task_struct *p, u64 enq_flags)
{
    // task_classes is a eBPF map which stores task_classes keyed by pid
    struct task_class *cls = bpf_map_lookup_elem(&task_classes, &p->pid);

    if (!cls) {
        struct luasched_result result = {
            .dsq      = DSQ_DEFAULT,
            .slice_ns = SCX_SLICE_DFL,
        };
        // if no class found, call Lunatik
        bpf_luasched_run(p, &result);

        struct task_class new_cls = {
            .dsq      = result.dsq,
            .slice_ns = result.slice_ns,
        };
        // update the map for future scheduling
        bpf_map_update_elem(&task_classes, &p->pid, &new_cls, BPF_ANY);
        scx_bpf_dispatch(p, result.dsq, result.slice_ns, 0);
        return;
    }

    scx_bpf_dispatch(p, cls->dsq, cls->slice_ns, 0);
}

void BPF_STRUCT_OPS(luasched_exit_task, struct task_struct *p, struct
scx_exit_task_args *args)
{
    bpf_map_delete_elem(&task_classes, &p->pid);
}

SCX_OPS_DEFINE(luasched_ops,
    .enqueue   = (void *)luasched_enqueue,
    .dispatch  = (void *)luasched_dispatch,
    .exit_task = (void *)luasched_exit_task,
    .init      = (void *)luasched_init,
    .name      = "luasched");
```

3. Lua callback

```
local sched = require("sched")

local REALTIME = 0
local BATCH    = 1

local policy = {
    { pattern = "^nginx",  dsq = REALTIME, slice = 1000000  }, --  1ms
    { pattern = "^ffmpeg", dsq = BATCH,    slice = 10000000 }, -- 10ms
```

```
}

sched.attach(function(task)
    for _, rule in ipairs(policy) do
        if task.comm:match(rule.pattern) then
            return rule.dsq, rule.slice
        end
    end
    return sched.dsq.DEFAULT, sched.slice.DEFAULT
end)
```

4.3.7. Stretch Goal: BCC integration

The proposed plan makes eBPF and Lua complementary. The layer involving eBPF cannot be entirely replaced but we could think of generating the necessary eBPF program via Lua through BPF Compiler Collection (BCC).

There are existing BCC/Lua bindings but they mostly use old `libbcc` library and will require changes to be able to generate the needed kfuncs. Also these bindings target tracing subsystems like kprobe.

If the deliverables mentioned in this proposal are ready before time, we could start digging into exisitng BCC support for Lua and potentially extending it for Lunatik's use case. The big picture is to give the user an option to write eBPF programs via Lua bindings.

4.4. Benchmarks and Evaluation

I have tried writing a TC implementation similar to XDP in `sneaky-potato/luatc` branch of the project. Link here.

I evaluated two scenarios:

- A pure eBPF program returning `TC_ACT_OK` for each packet (baseline fast path)
- An eBPF program invoking `bpf_luatc_run`, which executes a Lua script returning `tc.action.OK`

I attached the following kprobe (this will measure the latency to *classify* packets) to measure latency in both the cases.

```
sudo bpftrace -e '
kprobe:tcf_classify { @start[tid] = nsecs; }
kretprobe:tcf_classify {
    if (@start[tid]) {
        @lat = hist(nsecs - @start[tid]);
        delete(@start[tid]);
    }
}'
```

The results:

Percentile	Pure eBPF	eBPF + Lua TC
p50	3µs	6µs

Percentile	Pure eBPF	eBPF + Lua TC
p95	6µs	12µs
p99	20µs	40µs

This hints to use Lua only when required, on slow paths.

Similar benchmarking needs to be performed with all the demonstration scripts.

4.5. Expected Results

At the end of this project the Lunatik ecosystem will gain:

- A reusable `lunatik_bpf_run()` abstraction layer
- A new luatc binding for Traffic Control
- A new luasched binding for Linux Extensible Scheduler class
- A Lua module for interacting with eBPF maps
- Documentation and working examples demonstrating the system

This will significantly expand Lunatik's ability to script kernel behavior using Lua.

5. Project Timeline

Period	Milestones
Community Bonding	Agree on the lunatik_bpf_run interface with mentors, set up a test VM with BTF-enabled kernel
Week 1–2	Implement core `lunatik_bpf_run()` infrastructure and refactor the existing `bpf_luaxdp_run()` implementation to use it
Week 3-5	Implement `luatc` binding and expose `__sk_buff` context to Lua.
Week 6-8	eBPF maps module.
Week 9-10	Implement `luasched` binding and expose `task_struct` context to Lua.
Week 11	Benchmarks of eBPF + Lua scripts.
Week 12	Polish, code cleanup, final evaluation.

6. Midterm Deliverables (After 5 Weeks)

By the midterm evaluation:

- Generic lunatik_bpf_run() layer implemented
- bpf_luaxdp_run() refactored to use the new infrastructure
- Basic luatc binding implemented
- Lua handlers capable of interacting with TC packet context
- Initial demonstration scripts completed

7. Final Deliverables

At the end of the project:

- Fully functional luatc module
- Fully functional luasched module
- eBPF maps Lua API
- Examples and documentation

8. Risks

While the full scope is achievable, the project is structured so that each subsystem builds incrementally on the previous one. The `lunatik_bpf_run` and `luatc` bindings serve as the primary production-ready deliverables, and subsequent component (`luasched`) reuses the same infrastructure, ensuring partial progress always results in usable contributions.

9. Future Scope

1. **Target more subsystems with generic bpf layer**.

The most natural extensions are

- **BPF_PROG_TYPE_CGROUP_SKB** for per-container network policy
- **BPF_PROG_TYPE_SOCKET_FILTER** for application-layer filtering with dynamic per-socket allow lists.
- Tracing program types (**BPF_PROG_TYPE_KPROBE**, **BPF_PROG_TYPE_TRACEPOINT**)

2. **Add more to Maps module**.

Extending coverage to LRU hash maps (automatic eviction without manual cleanup), array maps, and **BPF_MAP_TYPE_RINGBUF** for event streaming would significantly expand what Lua policy handlers can express.

3. **Complete BCC frontend**.

BCC supports Python and Lua bindings for loading eBPF programs. At the time of writing this document BCC-Lua does not support kfuncs and subsystems other than krobes and tracing. Most of the exposed helpers are tracking old libbcc API.

A natural extension of this project could be writing a proper BCC frontend for Lua which bridges the proposed plan in this document. This will reduce friction of writing separate eBPF programs and enable the user to control everything from Lua.

10. Why This Project

I have always been interested in how operating systems work and how we can tweak certain parts to get our job done. I find Lunatik very interesting in the sense that it tries to achieve something bold: kernel scripting with a high level language.

Lunatik helped me understand how Lua is actually a glue language for already running systems. It is lightweight, embeddable, and expressive enough to implement the dynamic logic.

Instead of building isolated bindings, the proposed infrastructure allows future Lunatik modules across multiple subsystems to reuse the same mechanism, multiplying the scripting capabilities of the kernel.